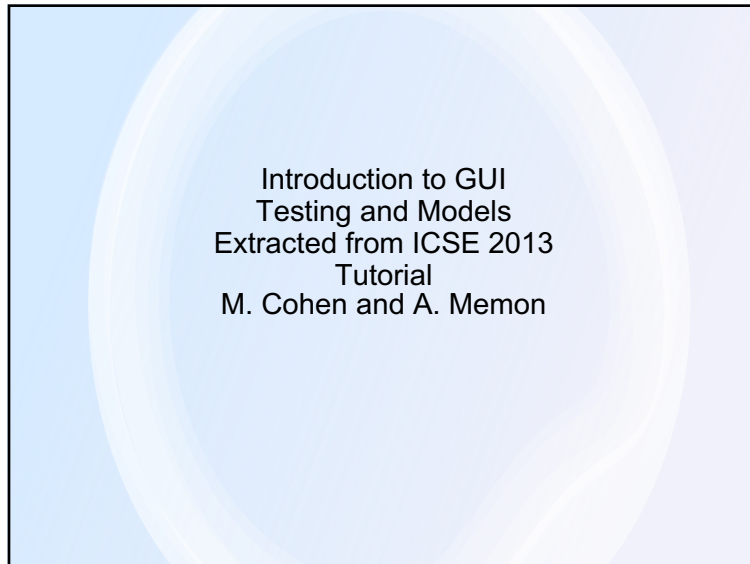
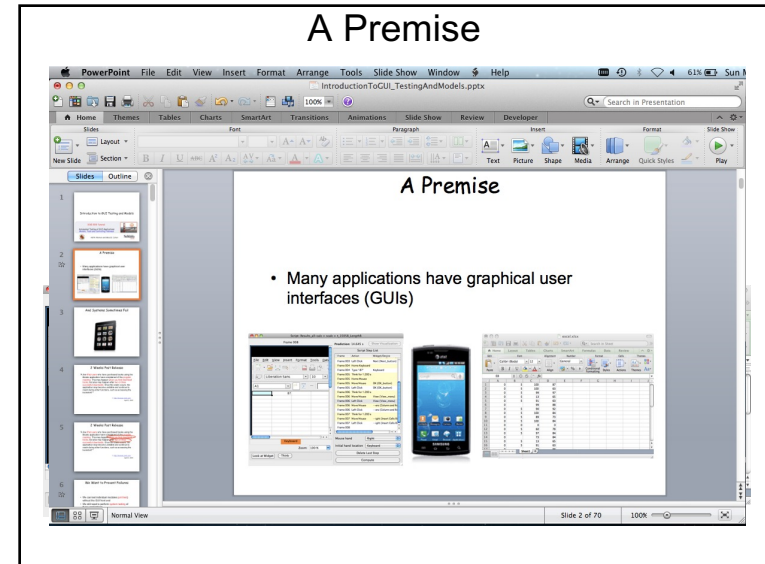




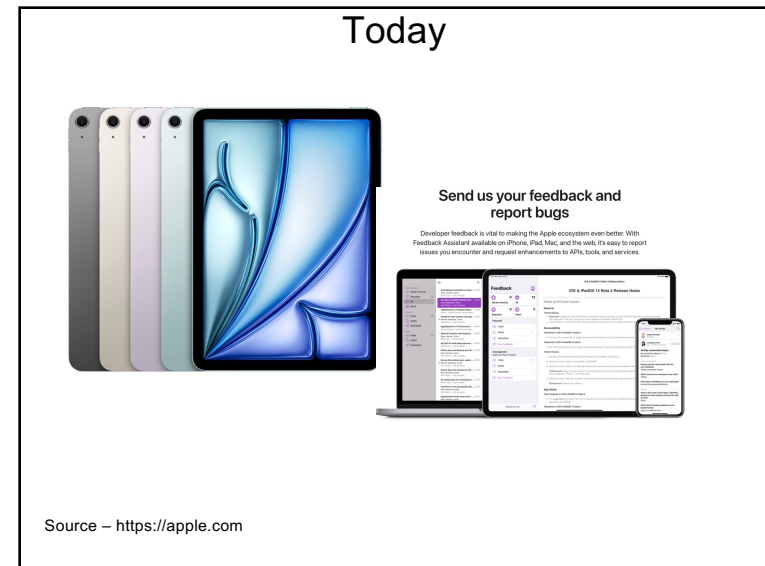
1



2



3



4

2 Weeks Post Release

"A few iPad users who have purchased books using the iBooks application have complained of the program crashing. This may happen when you first download books, but also may happen after two or three successful downloads. Once the crash occurs, the application may become unstable and continue to crash during other functions, such as accessing the bookshelf."¹

1. <http://reviews.cnet.com/>

April 5, 2010

5

2 Weeks Post Release

"A few iPad users who have purchased books using the iBooks application have complained of the program crashing. This may happen when you first download books, but also may happen after two or three successful downloads. Once the crash occurs, the application may become unstable and continue to crash during other functions, such as accessing the bookshelf."¹

1. <http://reviews.cnet.com/>

April 5, 2010

6

We Want to Prevent Failures

- We can **test individual modules** (unit test) without the GUI front end
- We still need to **perform system testing** of these applications
 - Performed **through the GUI**
 - **Test cases are sequences** of events (e.g. click/touch button or widget, rotate)

7

GUI System Testing Can Be Effective

- GUI testing can uncover faults in:
 1. GUI code
 2. Glue code
 3. **Underlying business logic**

[Brooks, Robinson, Memon, An Initial Characterization of Industrial Graphical User Interface Systems" ICST 2009]

8

But it is
Challenging

- Behavioral/interface specifications are often **not well defined**
- Have an **enormous** potential **event-space**
- **Platform specificity**
- **Need automation!**

9

Automated GUI Testing

- **Random actions** on the GUI
- Or:
- Create a **model of the GUI**
 - **State machine** models
 - **Graph model** (abstracts out some behavior)

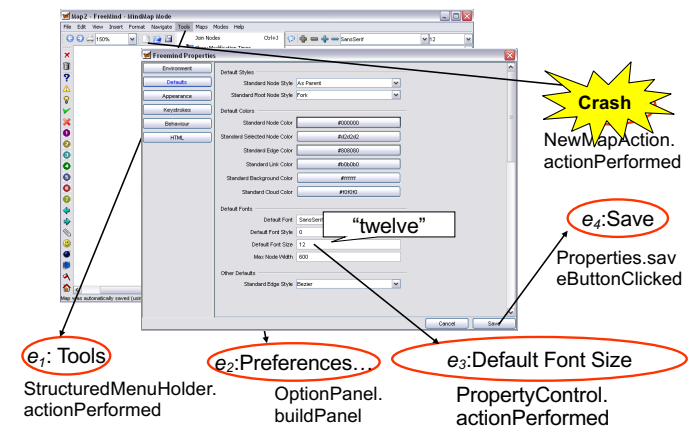
10

In this
Lecture

- I will present an overview of **model-based** GUI Testing

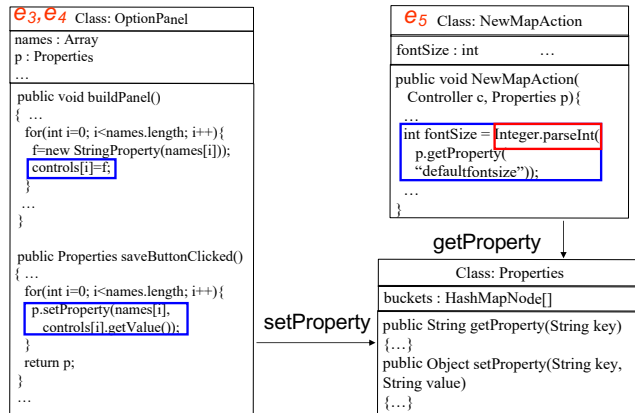
11

Typical System GUI Test



12

Associated Code



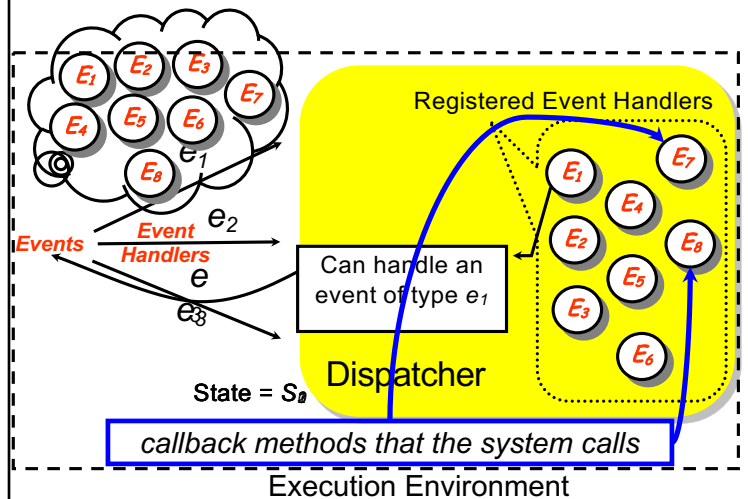
13

Complexity of GUI Testing

- GUI faults surface with **precise event sequences**
 - Drive the software into **particular states**
- Multiple components/classes** involved
 - Static analysis does not scale
- Source** for all parts of GUI **not always available**
 - Library code
- GUI is **"big integrator"** of software
 - It brings together all code
 - Many times these code pieces are never tested together

14

GUI Software in Action!



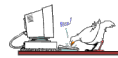
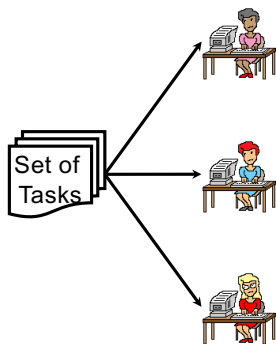
15

GUI Testing is Complex!

- Hard to do system testing**
 - multi-language, reflection, callbacks and multi-threading, etc.
- GUIs impose **work-flows over components** (event handlers)
- GUI event sequences **trigger component interactions**
- GUI testing = integration testing of event handlers
- GUI test case = sequence of events**
- Huge input and state space

16

How GUIs are Tested – Manual



17

How GUIs are Tested – Code Tests



```
public void testLoginScreen() {
    ...
    // (1) type in "qing" into the login name textbox, and "whatever" to the password textbox
    getHelper().sendKeys( new StringEventData( this, userNameField, "qing" ) );
    getHelper().sendKeys( new StringEventData( this, passwordField, "whatever" ) );

    // (2) click on "Enter" button
    getHelper().enterClickAndLeave( new MouseEventData( this, enterButton ) );

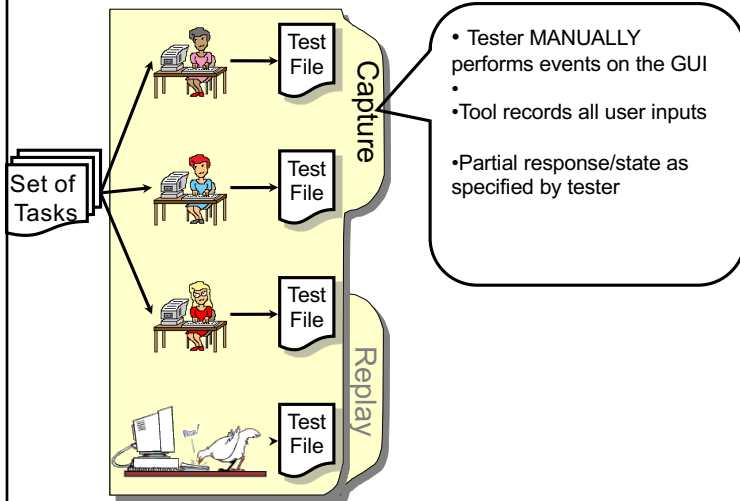
    // (3) waiting for response
    DialogFinder dFinder = new DialogFinder( null );
    dFinder.waitFor( 0 );

    // (4) login screen window is disposed
    showingDialogs = dFinder.findAll( loginScreen );
    assertEquals( "Number of dialogs showing is wrong", 0, showingDialogs.size() );
    assertTrue( "Login screen is showing still", !loginScreen.isShowing() );

    // (5) main GUI window shows up
    FrameFinder fFinder = new FrameFinder( null );
    showingWindows = fFinder.findAll();
    assertEquals( "Number of windows showing is wrong", 1, showingWindows.size() );
    ...
}
```

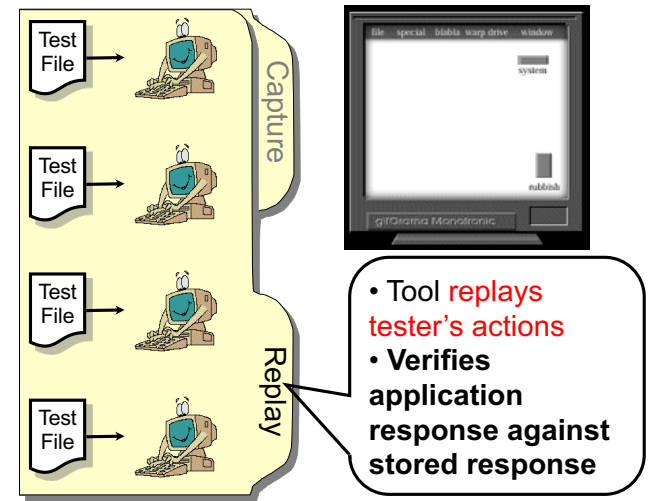
18

How GUIs are Tested – Capture/Replay

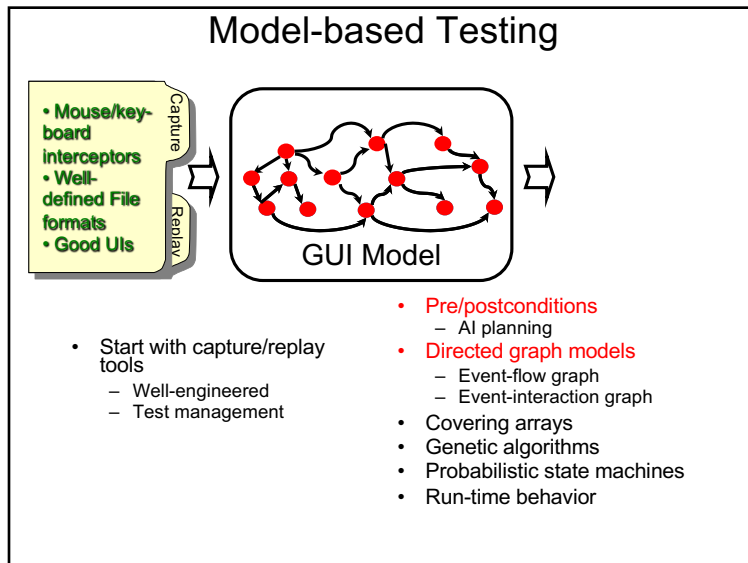


19

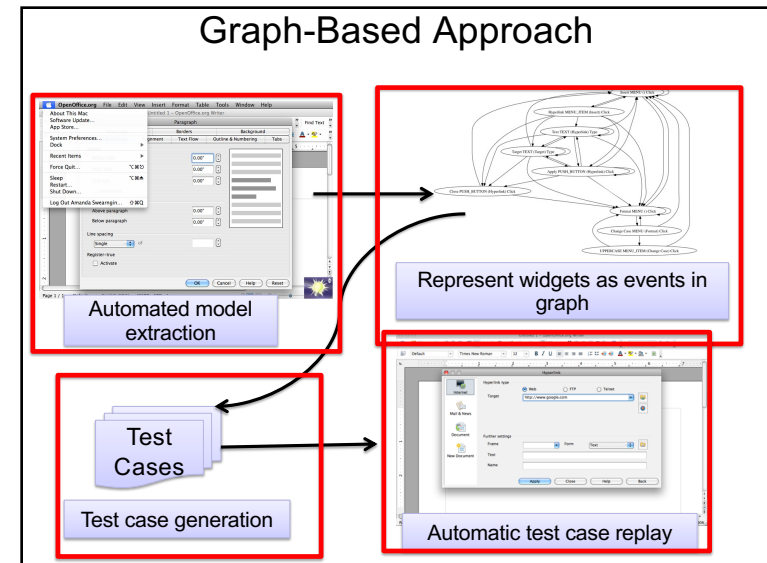
How GUIs are Tested – Capture/Replay



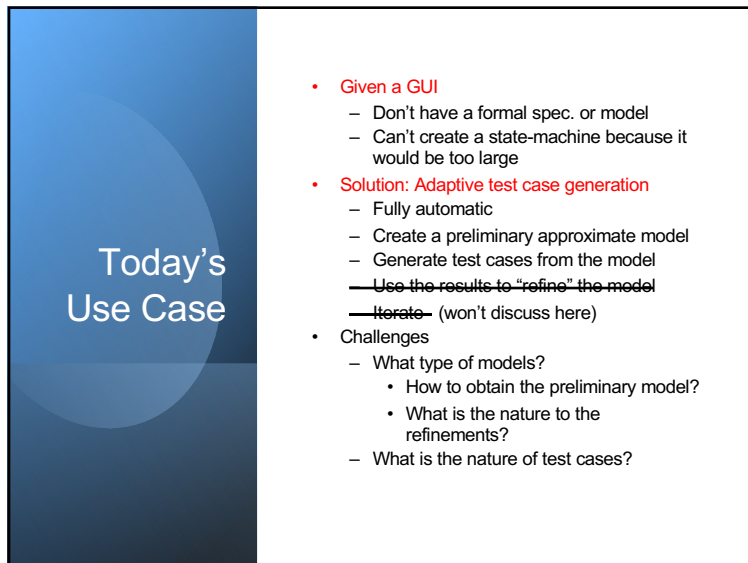
20



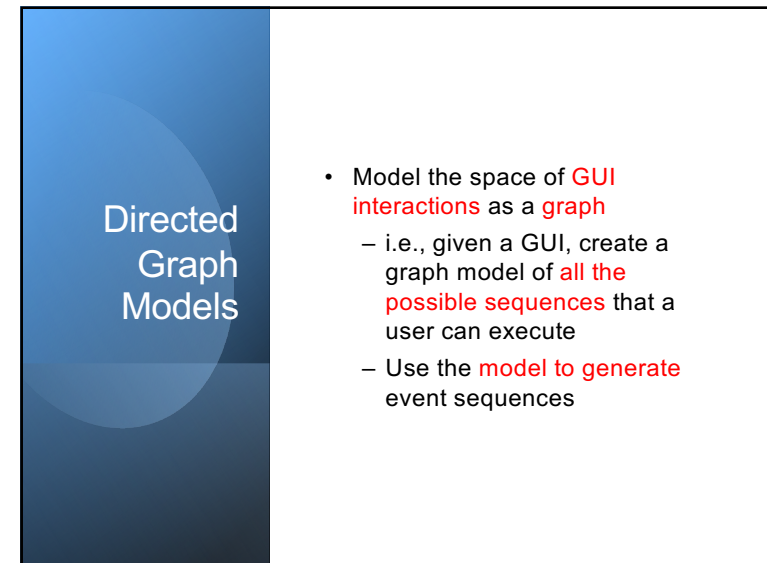
21



22



23



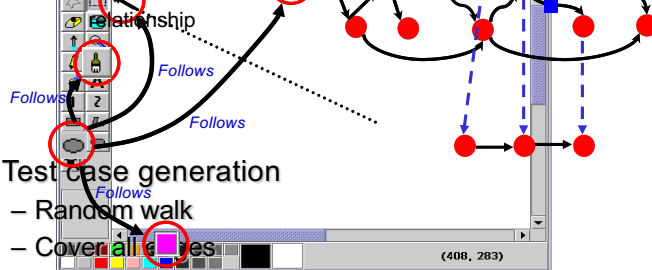
24

Sampling The Event-Interaction Space

- Event flow graph (EFG)

- Nodes: all GUI events

- Edges: Follows

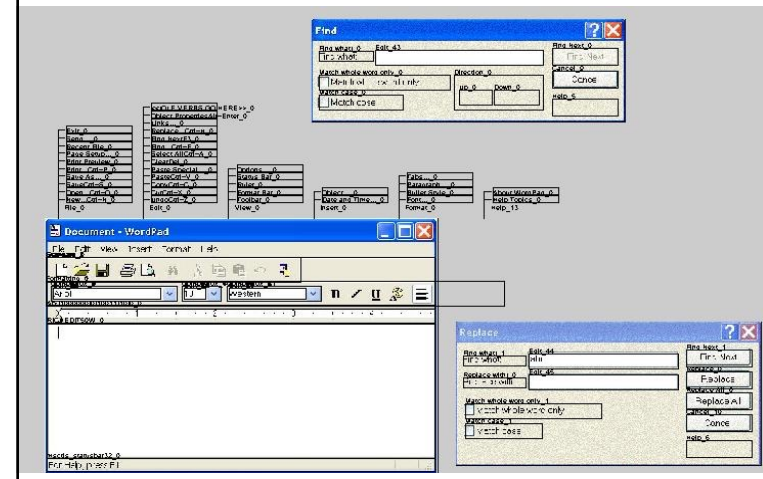


- Test case generation

- Random walk
- Cover all edges

25

A Part of MS WordPad



26

Modeling Event Interactions

- Event flow model goal:
 - Represent all possible event interactions on the GUI
 - This will allow a model that can be traversed for test case generation
- Graph model:
 - Vertex represents an event (click-on-edit, click-on-paste)
 - An edge from vertex x to vertex y shows that an event y can be performed immediately after event x
 - y follows x

27

Graph model

- Analogous to CFG (control flow graph)
- A state machine model equivalent to this graph can be constructed; state would represent possible events that can be executed on the GUI, transitions cause state changes
- In general a GUI that has no restrictions on event ordering and no windows/menus would have a fully connected graph; in practice this is not the case

28

More on the Graph Model

- **Modal windows:**
 - A GUI window that once invoked monopolizes the GUI interaction with user/program
 - Restricts focus of the user to a specific range of events within the window until terminated
- **Modeless windows:**
 - Do not restrict events; instead they expand the possible set of events available to the user

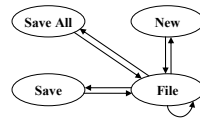
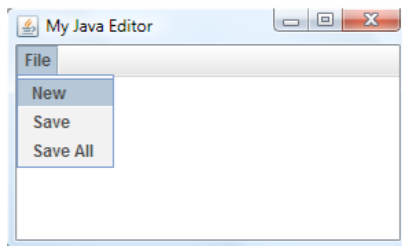
29

Event Flow Graph

- Event interactions within a modal dialogue can be represented as an **event-flow graph (EFG)**
- An EFG represents **all possible event sequences that can be executed in a dialogue**
- Once all of the **modal dialogues of a GUI have been modeled as EFGs** then the **interactions between EFGs can be identified and modeled**
- Integration tree: identifies interactions (invocations) between modal dialogs

30

Example EFG



31

Obtaining the EFG

- To construct an EFG first one must **identify modal and modeless windows** and their "invokes" relationships
- Classification of GUI Events:
 - Restricted-focus events
 - Open modal-windows
 - Un-restricted focus events
 - Open modeless windows
 - **Termination events**
 - Close modal windows

32

Obtaining the EFG

- **Menu-open events:**
 - Used to open menus
- **System interaction events:**
 - Not used to manipulate structure of the GUI; used to interact with underlying software to perform some action
- **Algorithm to construct EFG:**
 - Uses event classifications to construct the set of follows for each event

33

Algorithm for Follows

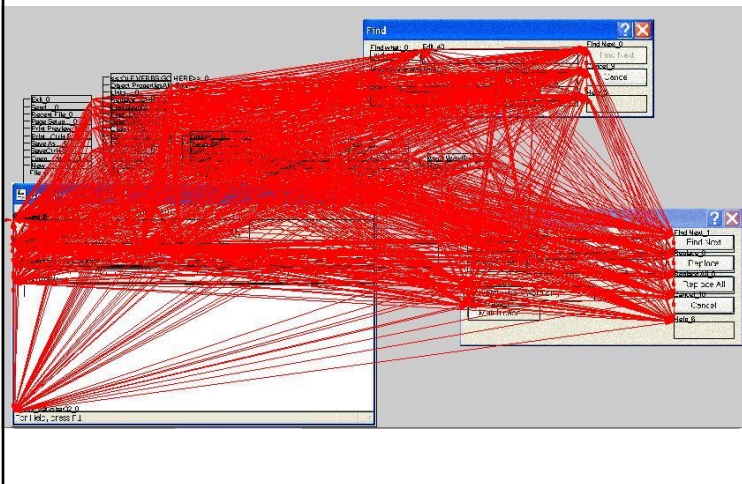
```

ALGORITHM: GetFollows(
v: Vertex or Event){
  IF EventType(v) = menu-open
  IF  $v \in B$  of the modal dialogue that contains v
    return(MenuChoices(v)  $\cup$  {v}  $\cup$  B)
  ELSE
    return(MenuChoices(v)  $\cup$  {v}
       $\cup$  follows(parent(v)));
  IF EventType(v) = system-interaction
    return(B);
  IF EventType(v) = termination
    return(B of Invoking modal dialogue);
  IF EventType(v) = unrestricted-focus
    return(B  $\cup$  B of Invoked modal dialogue);
  IF EventType(v) = restricted-focus
    return(B of Invoked modal dialogue);
}
  
```

Figure 1. Computing follows(v) for a vertex v.

34

Its Event-Flow Graph



35

Automation

- How do we create the event-flow graph?
 - **Create an approximation automatically**
 - Reverse Engineering
 - GOAL: Obtain the Event-Flow Graph
 - Fully automatically
 - No source code

36

Reverse Engineering – GUI Ripping

- **Dynamic algorithm**
 - No need for source code
- Execute the GUI-based software
 - **Traverse the GUI**
 - Obtain handle of first window
 - **Use windowing API to extract widgets/menus**
- **Apply transformations**
 - Based on GUI dialogs
 - GUI hierarchy
 - Enabled/disabled widgets
- Traverse multiple times if needed
- Engineering Issues
 - Understanding platform-specific GUI frameworks
 - OS-specific GUI handling
 - Introspection
 - Windowing API
- Result – Generic process for GUI Ripping
- MS Windows, Java Swing, Openoffice (UNO - Universal Network Objects), iOS, Android, Web (Selenium), Accessibility API
- Immediate impact – Obtained an approximate EFGs for large GUIs in a few minutes

37

Why is it an Approximate Model?

- **GUI Ripper performs one traversal to obtain all event-flows**
 - Depth-first (breadth-first for mobile platforms)
 - Miss events/windows
 - Exit application prematurely
 - State-based relationships
 - Conservative algorithm
 - Some event-sequences might be un-executable
- **IF we want a better approximation**
 - Bootstrap the Ripper
 - Ripper traversal customization
 - Iterative process

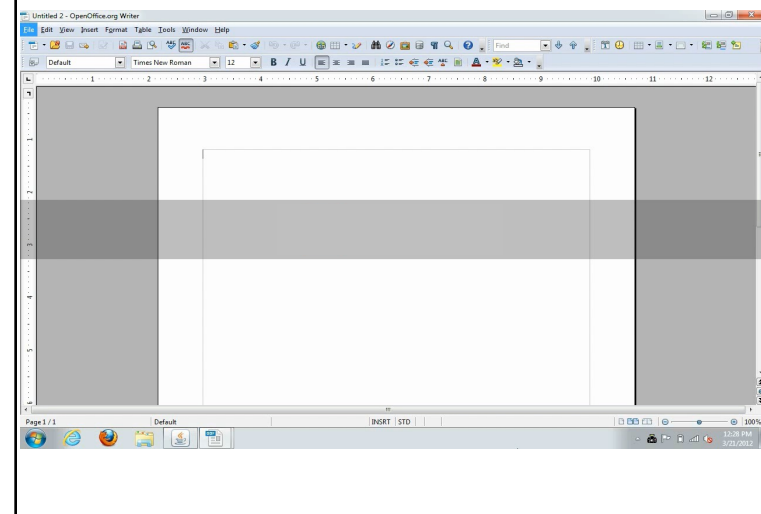
38

Impact of GUI Ripper

- **A way to generate test cases for large GUIs**
 - Random, Event-flow graph edge adequate, Code-coverage adequate, Covering arrays
 - Detected several faults in fielded software
- **Millions of test cases**
 - Dedicated 120 dual-CPU machine cluster
 - CONDOR jobs on UMIACS clusters (1000+ machines)
- For us, this enabled experimentation
 - Examine execution results to better understand the nature of GUI software

39

Extracting (Ripping) the Model



40

Extracting (Ripping) the Model

41

EFG for Part of ArgoUML

42

EFG for OpenOffice Writer

The diagram illustrates the EFG for OpenOffice Writer, showing a central 'Format' cluster and several peripheral clusters connected by numerous edges. The central cluster is orange and contains many sub-functions. The peripheral clusters are color-coded: purple for 'Styles and Formatting', green for 'Navigation', blue for 'New Document From Template', yellow for 'Change Case', blue for 'Original', and green for 'Page'. The edges represent the relationships between these functions.

43

Test Generation

- A GUI test case:
 - $\langle S0: e_1, e_2, \dots, e_n \rangle$ is a sequence where $S0$ is the start state from which this sequence is executed
 - Can use the graph model to traverse states via the follows edges
 - (capture replay is doing this manually)
- Can obtain a notion of coverage of the Event space

- A GUI test case:
 - $\langle S_0: e_1, e_2, \dots, e_n \rangle$ is a sequence where S_0 is the start state from which this sequence is executed
 - Can use the graph model to traverse states via the follows edges
 - (capture replay is doing this manually)
- Can obtain a notion of coverage of the Event space

Test Case Generation

- Use the event-flow graph to:
 - cover all edges (length 2 tests)
 - cover all length M events (where M is the number of nodes)
 - randomly generate tests of length M

45

Operator execution for oracles

- GUI oracle:
 - **Execution monitor** that extracts **the state of the GUI** using reverse engineering as a test case is executed
 - Oracle procedure that uses set equality compares the actual state with the oracle information (its expected state)
 - The oracle information for test case $\langle S_0: e_1, e_2, \dots, e_n \rangle$ is defined as a sequence of states $S_1; S_2; \dots; S_n$ such that S_i is the expected state of the GUI after e_i is executed

46

Obtaining Oracle Information

- Event flow models represent events as state transducers. Uses pre-conditions.
- Can compute the expected state S_i if have information from state S_{i-1} and the operator representing event e_i via additions and deletions to the list of properties representing S_{i-1} .
- Can detect certain kinds of failures:
 - Deviations from design/and or specifications

47

More on Oracles

- **Oracle information** – expected output
- **Oracle procedure** – compares expected with actual
- Example in Spreadsheet:
 - Oracle information:
 - (a) expected value of cell
 - (b) expected value of all cells
 - Oracle procedure:
 - (a) check for equality
 - (b) determine if value falls within certain range

48

Oracle Example

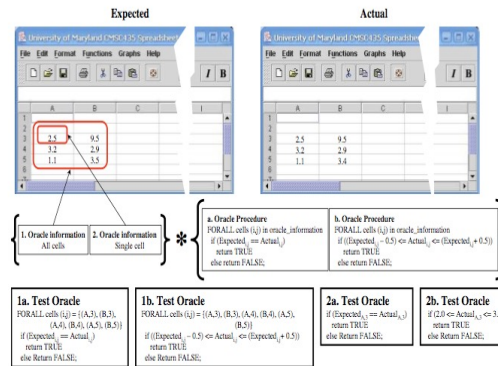


Fig. 1. Four different types of oracles for a spreadsheet program.

49

GUI Oracle Information

- Widget
- Active window
- All windows
(increasing detail/cost)

50

GUI Oracle Procedure

- Check equality of widget/active windows/all windows after each event
- Check for equality of widget/active windows/all windows after the last event

51

Oracle Details

- Implemented with **Assert Statements** containing:
 - Expected values of certain properties of widgets
 - Comparison process (which is equality) and frequency of execution
- Expected values are generated from a formal specification or from a baseline version that is assumed to be correct
- Current values obtained as program executes

52

Oracle Example

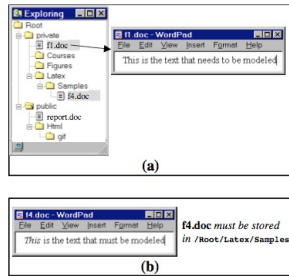


Figure 4: The Action Sequence of Figure 3 Transforms the GUI from: (a) the Initial State, to (b) the Final State

```

initial
contains(not private)
contains(private LaTeX)
contains(Latex Samples)
contains(file(Samples f1.doc))
contains(file(private f1.doc))
currentFont(Times Normal 12pt)
in(f1.doc "This")
font("This" Times Normal 12pt)
in(f1.doc "is the")
font("is the" Times Normal 12pt)
in(f1.doc "text")
font("text" Times Normal 12pt)
in(f1.doc "that")
font("that" Times Normal 12pt)
in(f1.doc "needs to")
font("needs to" Times Normal 12pt)
in(f1.doc "be modeled")
font("be modeled" Times Normal 12pt)

```

Figure 5: Representing the Initial State.

```

Operation Name
OpenGL: DIRS, file: FILES
Preconditions
contains(dir, file)
Effects
/* The current file is now file */
currentFile(file)
/* Now there are no objects on the screen */
~onScreen(obj) V obj: OBJECTS
/* All objects in FILE are now on the screen */
onScreen(obj) V obj: OBJECTS in(FILE, obj)

```

Example

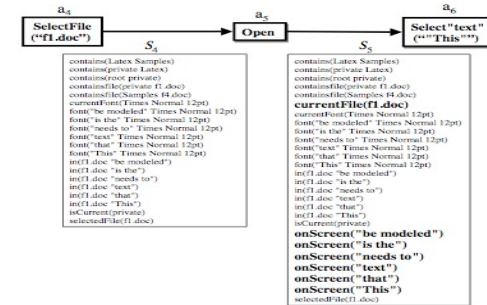


Figure 7: A Few Test-Case Actions with Expected State Information.

GUITAR

- The tool (called GUI Testing Framework or GUITAR) (<https://sourceforge.net/projects/guitar/>) was developed by A. Memon and his students at the University of Maryland. The following paper describes its architecture:
- [GUITAR: an innovative tool for automated testing of GUI-driven software](#) by BaoN. Nguyen, Bryan Robbins, Ishan Banerjee, and Atif Memon, *Automated Software Engineering*, 2013, pp. 1-41.
- YouTube demo <https://www.youtube.com/watch?v=dMr12bzNSxM>